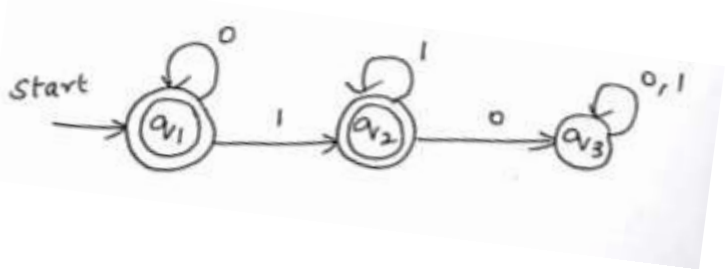
SAVEETHA SCHOOL OF ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING

CSA13 – THEORY OF COMPUTATION

ASSIGNMENT – 2

		CO	BTL
1	Write closure properties of regular languages with respect to compliment, kleen star, reversal and substitution	2	4
2	Construct a Non Deterministic Finite automata with ε-transitions to represent the language denoted by the regular expression (a+b)*(ab)*	2	3
3	Give an informal description of the language denoted by the regular expression (0+1)*00	2	3
4	Justify that the language L consisting of all strings with an equal number of a's and b's over the alphabet **Σ**={a,b} is not regular using pumping lemma .	2	4
5	Apply Arden's Theorem and derive the regular expression for the language represented by the given automata 	2	3

SOLUTIONS

Answer 1: Closure properties of regular languages

Idea. A class of languages is *closed* under an operation if applying the operation to languages in the class always gives a language that is still in the class. Regular languages are closed under all four operations asked here. In each case we can show it by building a new finite automaton (or regular expression) from the old one.

Operation	Closed?	How the new language is obtained
Complement	Yes	Make the DFA complete, then swap final and non-final states
Kleene star	Yes	Add a new start state and ε-moves that loop the final states back to the old start
Reversal	Yes	Reverse every arrow, swap start and final states
Substitution	Yes	Replace each symbol a by a regular language s(a) in the regular expression

Table 1: Summary of closure properties

1. Complement

Let L be accepted by a complete DFA $M = (Q, \Sigma, \delta, q0, F)$. Define $M' = (Q, \Sigma, \delta, q0, Q - F)$. A string w ends in a state of F in M exactly when it ends outside F in M', so M' accepts $\Sigma^* - L$. Hence the complement of a regular language is regular. *Example:* if L is the set of strings ending in 00, the complement is the set of strings that do not end in 00.

2. Kleene star

Let N be an NFA for L. Build N' with a new start state s that is also final (so ε is accepted). Add an ε-move from s to the old start state, and an ε-move from every old final state back to the old start state. Any string in L* is a concatenation of strings of L, and N' can run through them one after another, so N' accepts L*. *Example:* L = {ab} gives L* = (ab)*.

3. Reversal

Let N be an NFA for L with one final state (add ε-moves to a new single final state if needed). Reverse the direction of every transition, make the old final state the new start state and the old start state the new final state. The new machine accepts w^R exactly when N accepts w. *Example:* L = ab* gives L^R = b*a. In terms of regular expressions, $(r_1 r_2)^R = r_2^R r_1^R$, $(r_1 + r_2)^R = r_1^R + r_2^R$ and $(r^*)^R = (r^R)^*$.

4. Substitution

A substitution s maps each symbol a of Σ to a regular language s(a). For a regular expression r, replace every symbol a in r by a regular expression for s(a). Since union, concatenation and star are preserved, the result is again a regular expression, so s(L) is regular. *Example:* take L = 0*1 with s(0) = a+b and s(1) = cc. Then s(L) = (a+b)*cc, which is regular.

Answer 2: ε-NFA for the regular expression (a+b)*(ab)*

Idea. The expression is a concatenation of two parts. The first part (a+b)* is a loop that reads any number of a's and b's. The second part (ab)* is a loop that reads the block "ab" any number of times. An ε-transition joins the two parts, because no symbol is consumed when we move from the first part to the second. Both parts accept the empty string, so the state after the ε-move is final.

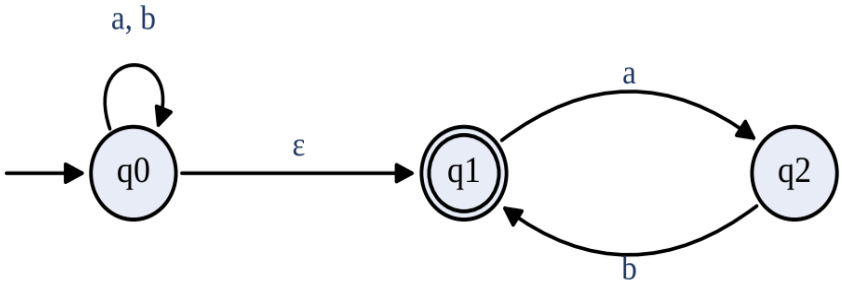


Figure 1: ε-NFA for (a+b)*(ab)*

Formal definition

N = (Q, Σ, δ, q0, F) with Q = {q0, q1, q2}, Σ = {a, b}, start state q0 and F = {q1}. The transition function is:

δ	a	b	ε
→ q0	{q0}	{q0}	{q1}
* q1	{q2}	{ }	{ }
q2	{ }	{q1}	{ }

Table 2: Transition table of the ε-NFA

Role of each state

State	Meaning
q0	Reading the (a+b)* part: any symbol keeps us here
q1	Between blocks of (ab)*; also the final state
q2	Just read an "a" of a block; a "b" must follow to return to q1

Table 3: Role of each state

Example: the string bab is accepted

q0 –b→ q0 –ε→ q1 –a→ q2 –b→ q1. The machine stops in q1, which is final, so **bab** is accepted (b comes from (a+b)* and ab from (ab)*). The string aa is also accepted: q0 –a→ q0 –a→ q0 –ε→ q1.

Note. Since (ab)* contains ε, the language (a+b)*(ab)* is the same as (a+b)*, which is every string over {a, b}. The ε-NFA above accepts exactly this, and it is built directly from the structure of the given expression.

Answer 3: Informal description of (0+1)*00

Idea. The part (0+1)* stands for any string of 0's and 1's, including the empty string. It is followed by exactly "00". So the expression describes every binary string that **ends with two consecutive 0's**. The shortest string in the language is 00.

Examples

String	In the language?	Reason
00	Yes	Prefix is ε, then 00
100	Yes	Prefix 1, then 00
0110100	Yes	Prefix 01101, then 00
1000	Yes	Prefix 10, then 00
0	No	Too short, needs two 0's
1001	No	Ends with 1
010	No	Last two symbols are 10

Table 4: Strings checked against (0+1)*00

An equivalent DFA

A DFA only has to remember how many 0's were read at the end of the input so far. Let s0 mean "no 0 at the end", s1 mean "exactly one 0 at the end" and s2 mean "at least two 0's at the end". The state s2 is final.

δ	0	1
→ s0	s1	s0
s1	s2	s0
* s2	s2	s0

Table 5: Transition table of the DFA

Check. For 1100: s0 –1→ s0 –1→ s0 –0→ s1 –0→ s2, which is final, so it is accepted. For 1001: the last symbol 1 sends s2 back to s0, so it is rejected. Both agree with the description.

Answer 4: L = {w : number of a's = number of b's} is not regular

Pumping lemma. If L is regular, there is a number $p \geq 1$ (the pumping length) such that every string s in L with $|s| \geq p$ can be written as $s = xyz$ where (1) $|xy| \leq p$, (2) $|y| \geq 1$ and (3) xy^iz is in L for every $i \geq 0$. To show a language is not regular we assume it is, and then find one string for which no split can satisfy all three conditions.

Proof by contradiction

- Assume L is regular and let p be its pumping length.
- Choose $s = a^pb^p$. It has p a's and p b's, so s is in L, and $|s| = 2p \geq p$.
- Let $s = xyz$ as given by the lemma. Because $|xy| \leq p$, both x and y lie inside the first p symbols, which are all a's. So $y = a^k$ for some $k \geq 1$.

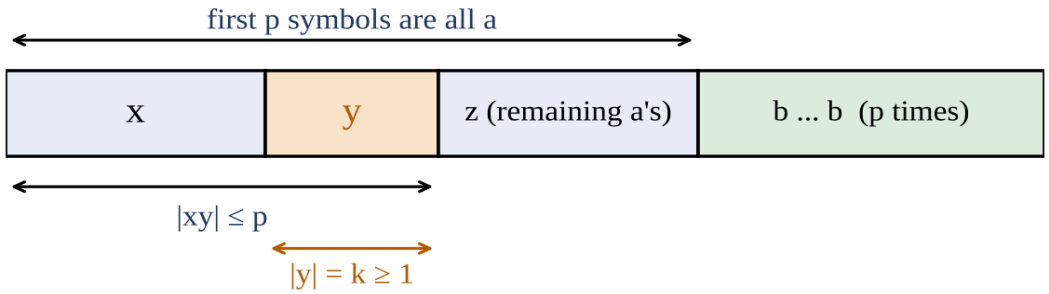


Figure 2: Every allowed split puts y inside the a-block

Pump y up or down

i	Pumped string xy^iz	Number of a's	Number of b's	In L?
0	$a^{p-k}b^p$	p − k	p	No
1	a^pb^p	p	p	Yes
2	$a^{p+k}b^p$	p + k	p	No

Table 6: Effect of pumping y

Take i = 2. The string $xy^2z = a^{p+k}b^p$ has p + k a's but only p b's. Since $k \geq 1$, the counts are different, so this string is not in L. This contradicts condition (3) of the pumping lemma.

Conclusion. The assumption that L is regular is false. Hence the language of all strings with an equal number of a's and b's is **not regular**. Intuitively, a finite automaton has only finitely many states and cannot count an unbounded number of a's to compare with the b's.

Answer 5: Regular expression of the automaton using Arden's Theorem

Arden's Theorem. If P and Q are regular expressions and P does not contain ε, then the equation R = Q + RP has the unique solution R = QP*.

Idea. Write one equation for every state, where the equation of a state is the union of (incoming state)·(symbol) over all arrows entering it, plus ε for the start state. Then solve the equations one by one using Arden's Theorem. In the given automaton q1 is the start state; q1 and q2 are taken as final states, and q3 is a trap state (once there, the input is never accepted).

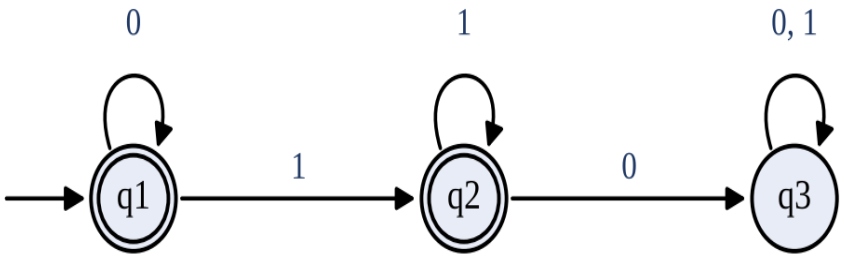


Figure 3: The given automaton (start q1, final q1 and q2)

Step 1: State equations

State	Equation	Arrows entering the state
q1	q1 = ε + q1·0	start state, loop on 0
q2	q2 = q1·1 + q2·1	from q1 on 1, loop on 1
q3	q3 = q2·0 + q3·(0+1)	from q2 on 0, loop on 0,1

Table 7: Equations of the states

Step 2: Solve for q1

q1 = ε + q1·0 has the form R = Q + RP with Q = ε and P = 0. By Arden's Theorem, q1 = ε·0* = **0***.

Step 3: Solve for q2

Put q1 = 0* into q2 = q1·1 + q2·1. This gives q2 = 0*1 + q2·1, which has the form R = Q + RP with Q = 0*1 and P = 1. So q2 = 0*1·1* = **0*11***.

Step 4: Expression for the language

The final states are q1 and q2, so the language is their union:

$$L = q1 + q2 = 0^* + 0^*11^* = 0^*(\varepsilon + 11^*) = \mathbf{0^*1^*}$$

since ε + 11* = 1*.

Check. The string 0011 goes q1 –0→ q1 –0→ q1 –1→ q2 –1→ q2, which is final, and 0011 matches 0*1*. The string 10 goes q1 –1→ q2 –0→ q3, a trap state, so it is rejected, and 10 does not match 0*1* (a 0 after a 1). So the language is all strings of 0's followed by 1's.